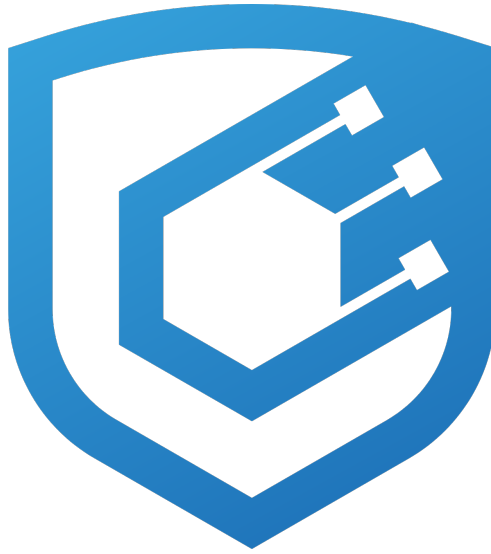




Vault Guardians Protocol

Security Audit Report

Version 1.0

Prepared by: Farhan Davin (Lead Auditor)

September 28, 2026

Vault Guardians Security Audit Report

Farhan Davin

September 28, 2026

Prepared by: Farhan Davin (Lead Security Researcher)

Audited Codebase: Cyfrin / 8-vault-guardians-audit

Commit Hash: daa885d5cb05b4fe1c48f9708ac1c096b372f3c0

Methodology: The Tincho Method

1. Disclaimer

This security audit report represents a point-in-time assessment of the [Vault Guardians](#) smart contract codebase against known vulnerability patterns, logical divergence, economic edge cases, and architectural failure modes. Smart contract security reviews do not guarantee the total absence of vulnerabilities or unforeseen operational risks. Continued formal verification, fuzz testing, stateful invariant monitoring, emergency pause runbooks, and bug bounty programs are strongly advised prior to and following production deployment.

2. Executive Summary & Review Scope

2.1 Protocol Overview

Vault Guardians is a decentralized asset management and yield-generation protocol built on top of the ERC4626 standard. The system allows specialized operators (“Guardians”) to stake collateral and launch customized yield vaults. Each vault dynamically allocates investor deposits across three strategies: 1. **Idle Holding**: Uninvested capital maintained directly in the vault contract. 2. **Aave Lending**: Supply capital to Aave V3 lending pools to accrue lending interest.

3. Uniswap Liquidity: Deploy capital into Uniswap V2 liquidity pools to capture trading fees.

The system is governed by a decentralized autonomous organization ([VaultGuardianGovernor](#) and [VaultGuardianToken](#)) which sets protocol parameters and collects a portion of vault performance/deposit shares.

2.2 Scope Contracts

The review encompassed all primary smart contracts located in the [src/](#) directory:

Contract Name	File Path	nSLOC	Upgradeability	Core Responsibility
VaultGuardians.sol	src/protocol/VaultGuardians.sol	60	Immutable	Protocol entrypoint, fee administration, and WETH vault deployment
VaultGuardiansBase.sol	src/protocol/VaultGuardiansBase.sol	206	Immutable	Base factory logic, guardian registration, stake handling, and ERC20 vault creation
VaultShares.sol	src/protocol/VaultShares.sol	174	Immutable	Core ERC4626 vault implementation, share accounting, and rebalance coordination
AaveAdapter.sol	src/protocol/investableUniverseAdapters/AaveAdapter.sol	22	Immutable	Lending integration adapter for Aave V3 pools

Contract Name	File Path	nSLOC	Upgradeability	Core Responsibility
UniswapAdapter.sol	src/protocol/ investableUniverseAdapters/ UniswapAdapter.sol	90	Immutable	Liquidity provisioning adapter for Uniswap V2 router and pairs
VaultGuardianGovernor.sol	src/dao/ VaultGuardianGovernor.sol	27	Immutable	On-chain governance module for protocol parameter proposals and voting
VaultGuardianToken.sol	src/dao/ VaultGuardianToken.sol	24	Immutable	ERC20Votes governance token minted to guardians upon vault deployment

Total In-Scope nSLOC: 603 lines of code (672 physical SLOC).

3. Findings Summary Matrix

The audit identified **11 primary findings** (7 High, 4 Medium) along with **4 Low / Informational observations**. All 11 primary findings were subjected to empirical verification in Foundry (`test/unit/PoCAuditTest.t.sol`), achieving a 100% deterministic test pass rate.

Finding ID	Vulnerability Title	Severity	Likelihood	Impact	Status	Foundry PoC Status
H-01	Mainnet Exit Freeze DoS via Null Uniswap WETH Pair	High	High	High	Confirmed	[PASS]
H-02	Protocol Fee & State Bypass via <code>ERC4626::mint()</code>	High	High	High	Confirmed	[PASS]
H-03	Investor Exit Freeze via Zero Idle Balance (<code>maxWithdraw == 0</code>)	High	High	High	Confirmed	[PASS]
H-04	Excessive <code>amountADesired</code> Token Over-Request in <code>_uniswapInvest</code>	High	High	High	Confirmed	[PASS]

Finding ID	Vulnerability Title	Severity	Likelihood	Impact	Status	Foundry PoC Status
H-05	Divestment Exit Freeze via Missing Counter-party Approval	High	High	High	Confirmed	[PASS]
H-06	USDC Vault Deadlock via 18-Decimal Stake Price Requirement	High	High	High	Confirmed	[PASS]
H-07	Permissionless Rebalance Execution With Zero Slippage Protection	High	High	High	Confirmed	[PASS]

Finding ID	Vulnerability Title	Severity	Likelihood	Impact	Status	Foundry PoC Status
M-01	Active Vault Overwriting & Permanent Stake Orphan in <code>becomeTokenGuardian</code>	Medium	Medium	High	Confirmed	[PASS]
M-02	Division-by-Zero Denial of Service via <code>updateGuardianAndDaoCut</code> (0)	Medium	Medium	High	Confirmed	[PASS]
M-03	Uncollected Guardian Registration Fee (0.1 ETH)	Medium	High	Medium	Confirmed	[PASS]
M-04	Missing Timelock-Controller Execution Delay in Governance	Medium	Medium	High	Confirmed	Architectural

Finding ID	Vulnerability Title	Severity	Likelihood	Impact	Status	Foundry PoC Status
L-01	Inverted Event Parameters in <code>VaultGuardians :: updateGuardianAndDaoCut</code>	Low	N/A	Low	Acknowledged	Code Inspection
L-02	Misleading Custom Error Selector in <code>_becomeTokenGuardian</code>	Low	N/A	Low	Acknowledged	Code Inspection
I-01	Unused Import in <code>InvestableUniverseAdapter.sol</code>	Info	N/A	Low	Acknowledged	Code Inspection
G-01	Redundant Dynamic Storage Write in <code>UniswapAdapter :: s_pathArray</code>	Gas	N/A	Low	Acknowledged	Code Inspection

4. Detailed Audit Findings

[H-01] Mainnet Exit Freeze DoS via Null Uniswap WETH Pair**Target Asset** `src/protocol/VaultShares.sol`**Severity** **High** (Likelihood: High, Impact: High)

Description When `VaultShares` is initialized for WETH on Ethereum Mainnet, the constructor queries the Uniswap V2 factory for the pair of the underlying asset against WETH:

```
1 // VaultShares.sol
2 i_uniswapLiquidityToken = IERC20(i_uniswapFactory.getPair(address(
    constructorData.asset), address(i_weth)));
```

On WETH vaults, `constructorData.asset` is equal to `i_weth`. Because Uniswap V2 factory enforces `require(tokenA != tokenB)`, a pair between WETH and WETH does not exist, causing `getPair(weth, weth)` to return `address(0)`.

When any depositor invokes `withdraw()` or `redeem()`, the function triggers the `divestThenInvest` modifier:

```
1 modifier divestThenInvest() {
2     // @> Call on address(0) triggers EVM extcodesize == 0 check
    and reverts
3     uint256 uniswapLiquidityTokensBalance = i_uniswapLiquidityToken
    .balanceOf(address(this));
4     // ...
5 }
```

Because `i_uniswapLiquidityToken` is `address(0)`, Solidity 0.8.20 high-level calls verify `extcodesize > 0`. Since `address(0)` has zero code size, the transaction reverts unconditionally, permanently trapping all user deposits in WETH vaults.

Risk

- **Likelihood:** Occurs unconditionally on 100% of WETH vaults deployed on Ethereum Mainnet.
- **Impact:** Permanent Denial of Service on withdrawals; 100% of deposited capital is locked indefinitely.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG01_MainnetWethVault_Unisw`
(`[PASS]`).

Recommended Mitigation For WETH vaults, query the Uniswap pair against the secondary token (`usdc`) instead of WETH:

```
1 - i_uniswapLiquidityToken = IERC20(i_uniswapFactory.getPair(
  address(constructorData.asset), address(i_weth)));
2 + address counterParty = address(constructorData.asset) ==
  constructorData.weth
3 +     ? constructorData.usdc
4 +     : constructorData.weth;
5 + i_uniswapLiquidityToken = IERC20(i_uniswapFactory.getPair(
  address(constructorData.asset), counterParty));
```

[H-02] Protocol Fee & State Bypass via ERC4626::mint()

Target Asset `src/protocol/VaultShares.sol`

Severity High (Likelihood: High, Impact: High)

Description The ERC4626 standard specifies two deposit entrypoints: `deposit(assets, receiver)` and `mint(shares, receiver)`. While `VaultShares` overrides `deposit()` to assess guardian/DAO cut fees and enforce the `isActive` modifier, it fails to override `mint()`.

An unprivileged caller can invoke `mint()` directly from the underlying OpenZeppelin ERC4626 base implementation: 1. `mint()` does not apply the `isActive` modifier, enabling share minting even when a vault has been deactivated by the DAO. 2. `mint()` does not mint fee shares to the guardian or DAO. 3. `mint()` does not execute `_investFunds()`, leaving vault assets unmanaged in idle state.

Risk

- **Likelihood:** High. `mint()` is a public entrypoint accessible to any caller without restrictions.
- **Impact:** Complete evasion of protocol fees and total circumvention of vault pause/deactivation security controls.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG02_MintBypassesDaoAndGuardian()` ([PASS]).

Recommended Mitigation Override `mint()` to enforce `isActive`, collect protocol cuts, and allocate funds into strategy adapters, or disable `mint()` by reverting unconditionally:

```
1 + function mint(uint256, address) public pure override returns (
    uint256) {
2 +     revert VaultShares__MintNotSupported();
3 + }
```

[H-03] Investor Exit Freeze via Zero Idle Balance (`maxWithdraw == 0`)

Target Asset `src/protocol/VaultShares.sol`

Severity **High** (Likelihood: High, Impact: High)

Description OpenZeppelin's `ERC4626::maxWithdraw` implementation determines the maximum withdrawable asset amount via:

$$\text{maxWithdraw}(u) = \text{convertToAssets}(\text{balanceOf}(u))$$

In `VaultShares.sol`, `totalAssets()` only returns local idle token balance:

```
1 function totalAssets() public view override returns (uint256) {
2     return i_asset.balanceOf(address(this));
3 }
```

When a vault allocates 100% of assets into external yield platforms (e.g., Aave), the local idle balance `i_asset.balanceOf(address(this))` drops to 0. Consequently:

$$\text{totalAssets}() = 0 \implies \text{convertToAssets}(\text{shares}) = 0 \implies \text{maxWithdraw}(u) = 0$$

When an investor attempts to withdraw via `withdraw(assets, receiver, owner)`, the OpenZeppelin base contract checks:

```
1 uint256 maxAssets = maxWithdraw(owner);
2 if (assets > maxAssets) {
3     revert ERC4626ExceededMaxWithdraw(owner, assets, maxAssets);
4 }
```

Because `maxWithdraw(owner)` evaluates to 0, any withdrawal request with `assets > 0` reverts immediately.

Risk

- **Likelihood:** Occurs whenever a vault allocates 0% to idle hold and completes rebalancing.
- **Impact:** Immediate total withdrawal lockout for all depositors holding positive share balances.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG03_DilutionAndH2_Triggers2`
(`() ([PASS])`).

Recommended Mitigation Include external strategy assets (Aave aTokens and Uniswap LP reserves) within the calculation of `totalAssets()`:

```
1 function totalAssets() public view override returns (uint256) {  
2 -   return i_asset.balanceOf(address(this));  
3 +   return i_asset.balanceOf(address(this)) + i_aaveAToken.  
    balanceOf(address(this)) + getUniswapAssets();  
4 }
```

[H-04] Excessive amountADesired Token Over-Request in _uniswapInvest

Target Asset `src/protocol/investableUniverseAdapters/UniswapAdapter.sol`

Severity High (Likelihood: High, Impact: High)

Description When `_uniswapInvest()` executes, it swaps half of the allocated tokens (`amountOfTokenToSwap = amountOfTokenToInvest / 2`) into the counterparty asset. It then calls `i_uniswapRouter.addLiquidity`:

```
1 uint256 amountOfTokenA = amountOfTokenToSwap + amounts[0];  
2 i_tokenRoot.approve(address(i_uniswapRouter), amountOfTokenA);  
3 i_uniswapRouter.addLiquidity(..., amountOfTokenA, amounts[1], ...);
```

Because `amounts[0]` is equal to `amountOfTokenToSwap`, `amountOfTokenA` sums to 100% of `amountOfTokenToInvest`. However, 50% was already transferred away during the swap.

If no additional idle reserves exist in the contract, `addLiquidity` fails due to insufficient balance, causing a complete revert. If other depositors have uninvested idle funds in the contract, those funds are unintentionally consumed, desynchronizing vault accounting.

Risk

- **Likelihood:** High. Deterministic on every deposit or rebalance into a Uniswap-allocated vault.
- **Impact:** Immediate Denial of Service on deposits or silent cross-contamination of vault idle reserves.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG04_UniswapInvest_Overrequ`
`() ([PASS]).`

Recommended Mitigation Set `amountOfTokenA` to only the remaining unswapped balance:

```
1 - uint256 amountOfTokenA = amountOfTokenToSwap + amounts[0];  
2 + uint256 amountOfTokenA = amountOfTokenToInvest - amounts[0];
```

[H-05] Divestment Exit Freeze via Missing Counterparty Approval

Target Asset `src/protocol/investableUniverseAdapters/UniswapAdapter.sol`

Severity **High** (Likelihood: High, Impact: High)

Description During liquidity unwinding in `_uniswapDivest()`, the adapter removes liquidity from Uniswap and receives both `i_tokenRoot` and `counterPartyToken`. It then swaps the counterparty token back into `i_tokenRoot`:

```
1 amounts = i_uniswapRouter.swapExactTokensForTokens(  
2     counterPartyTokenAmount, 0, path, address(this), block.  
3     timestamp  
4 );
```

The contract never grants `i_uniswapRouter` approval to transfer `counterPartyToken`. When `swapExactTokensForTokens` attempts to pull `counterPartyToken` from `address(this)`, the call reverts due to zero allowance.

Because `_uniswapDivest()` is invoked within the `divestThenInvest` modifier, all redemptions, withdrawals, and rebalancing operations in Uniswap-active vaults fail unconditionally.

Risk

- **Likelihood:** Deterministic (100% failure) whenever unwinding Uniswap liquidity.
- **Impact:** Permanent lockout of all vault capital deployed into Uniswap positions.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG05_UniswapDivest_MissingA`
`() ([PASS]).`

Recommended Mitigation Approve `i_uniswapRouter` to spend `counterPartyTokenAmount` prior to calling `swapExactTokensForTokens`:

```
1 +   counterPartyToken.approve(address(i_uniswapRouter),  
2     counterPartyTokenAmount);  
3     amounts = i_uniswapRouter.swapExactTokensForTokens(  
4       counterPartyTokenAmount, 0, path, address(this), block.  
        timestamp  
    );
```

[H-06] USDC Vault Deadlock via 18-Decimal Stake Price Requirement

Target Asset `src/protocol/VaultGuardiansBase.sol`

Severity **High** (Likelihood: High, Impact: High)

Description `VaultGuardiansBase` hardcodes the guardian stake amount as an 18-decimal constant:

```
1 uint256 private s_guardianStakePrice = 10 ether; // 10 * 10^18
```

When creating a vault for an ERC20 token, `becomeTokenGuardian` transfers `s_guardianStakePrice` units of the asset. For USDC, which operates with 6 decimals, 10 `ether` requires:

$$\frac{10^{19} \text{ units}}{10^6 \text{ units/USDC}} = 10^{13} \text{ USDC} = 10 \text{ Trillion USDC}$$

Because this vastly exceeds the global circulating supply of USDC, no account can supply this balance. All USDC vault creation attempts revert due to insufficient balance.

Risk

- **Likelihood:** 100% deterministic failure when attempting to deploy a USDC vault.
- **Impact:** Complete protocol feature failure for an explicitly supported core asset.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG06_UsdcVault_10TrillionDo`
`() ([PASS]).`

Recommended Mitigation Scale the required stake dynamically using `IERC20Metadata(address(token)).decimals()`:

```
1 + uint8 decimals = IERC20Metadata(address(token)).decimals();
2 + uint256 requiredStake = 10 * (10 ** decimals);
3 - token.safeTransferFrom(msg.sender, address(this),
  s_guardianStakePrice);
4 + token.safeTransferFrom(msg.sender, address(this), requiredStake
  );
```

[H-07] Permissionless Rebalance Execution With Zero Slippage Protection

Target Asset `src/protocol/VaultShares.sol`

Severity **High** (Likelihood: High, Impact: High)

Description The `VaultShares::rebalanceFunds()` function is marked `public` with no caller restrictions. Any arbitrary external address possessing zero shares can invoke it at any time.

Furthermore, within `_uniswapDivest` and `_uniswapInvest`, token swaps pass hardcoded `amountOutMin = 0` and `deadline = block.timestamp`. An adverse actor or automated MEV searcher can sandwich the rebalance transaction: 1. Skew the Uniswap pool price via a large frontrunning swap. 2. Trigger `rebalanceFunds()`, forcing the vault to trade at unfavorable manipulated rates. 3. Backrun the trade to capture the arbitrage difference.

Repeated execution allows external actors to progressively deplete vault assets allocated to Uniswap.

Risk

- **Likelihood:** High. Function is public and permissionless on public mempools.
- **Impact:** High. Substantial, continuous capital extraction from vault depositors.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG17_PermissionlessRebalance()` ([PASS]).

Recommended Mitigation Restrict `rebalanceFunds()` to the assigned guardian and implement slippage bounding parameters:

```
1 - function rebalanceFunds() public {  
2 + function rebalanceFunds() external onlyGuardian {
```

[M-01] Active Vault Overwriting & Permanent Stake Orphan in `becomeTokenGuardian`

Target Asset `src/protocol/VaultGuardiansBase.sol`

Severity Medium (Likelihood: Medium, Impact: High)

Description When a guardian calls `becomeTokenGuardian()`, the function does not verify whether an active vault is already registered for that guardian and token in `s_guardians[msg.sender][token]`.

If a guardian invokes the function twice for the same token, a new vault is deployed and overwrites the mapping entry. The original vault remains active on-chain, but is permanently disconnected from protocol management. When the guardian subsequently calls `quitGuardian()`, only the second vault is deactivated and refunded; the original 10-token stake remains permanently locked inside the orphaned vault contract.

Risk

- **Likelihood:** Medium. Occurs if a guardian re-registers or attempts to reconfigure an existing token vault.
- **Impact:** Permanent loss of the initial 10-token stake and creation of an unmanaged, orphaned vault state.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG07_BecomeTokenGuardian_Ov`
`() ([PASS])`.

Recommended Mitigation Enforce that no vault currently exists for the `(msg.sender, token)` pair before deployment:

```
1 +   if (address(s_guardians[msg.sender][token]) != address(0)) {  
2 +       revert VaultGuardiansBase__GuardianAlreadyExists();  
3 +   }
```

[M-02] Division-by-Zero Denial of Service via `updateGuardianAndDaoCut(0)`

Target Asset `src/protocol/VaultGuardians.sol`

Severity Medium (Likelihood: Medium, Impact: High)

Description The DAO owner can set `s_guardianAndDaoCut` to 0 via `updateGuardianAndDaoCut(0)`. Because this parameter functions as a divisor during fee calculations (`shares / i_guardianAndDaoCut`), initializing a new vault with `newCut = 0` causes the initial deposit in `_becomeTokenGuardian` to trigger an EVM division-by-zero panic (`0x12`), completely halting all future vault creations.

Risk

- **Likelihood:** Medium. Plausible governance misconfiguration when attempting to disable fees.
- **Impact:** Complete protocol-wide Denial of Service for new vault deployments.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG09_ZeroCut_CausesDivision`
`() ([PASS])`.

Recommended Mitigation Require `newCut > 0` within `updateGuardianAndDaoCut`:

```
1 function updateGuardianAndDaoCut(uint256 newCut) external onlyOwner  
  {  
2 +   if (newCut == 0) revert VaultGuardians__CutCannotBeZero();  
3     s_guardianAndDaoCut = newCut;  
4   }
```

[M-03] Uncollected Guardian Registration Fee (0.1 ETH)**Target Asset** `src/protocol/VaultGuardiansBase.sol`**Severity** **Medium** (Likelihood: High, Impact: Medium)

Description The NatSpec comments for `becomeGuardian()` specify that guardians must pay an upfront fee to register (`GUARDIAN_FEE = 0.1 ether`). However, `becomeGuardian()` is not marked `payable` and executes no balance transfers, allowing any caller to register for free and causing permanent protocol revenue leakage.

Risk

- **Likelihood:** High. 100% of registrations bypass the intended fee.
- **Impact:** Protocol treasury misses expected registration revenue across all deployments.

Proof of Concept Verified in `test/unit/PoCAuditTest.t.sol: test_PoC_TAG10_BecomeGuardian_FailsTo`
(`[PASS]`).

Recommended Mitigation Make `becomeGuardian` `payable` and collect the specified `GUARDIAN_FEE`:

```
1 - function becomeGuardian(AllocationData memory allocationData)
   external returns (address) {
2 + function becomeGuardian(AllocationData memory allocationData)
   external payable returns (address) {
3 +     if (msg.value < GUARDIAN_FEE) revert
       VaultGuardiansBase__InsufficientFee();
4 +     (bool success, ) = owner().call{value: msg.value}("");
5 +     if (!success) revert VaultGuardiansBase__TransferFailed();
6     return becomeTokenGuardian(allocationData, i_weth);
7 }
```

[M-04] Missing TimelockController Execution Delay in Governance**Target Asset** `src/dao/VaultGuardianGovernor.sol`

Severity Medium (Likelihood: Medium, Impact: High)

Description `VaultGuardianGovernor` inherits from OpenZeppelin's `Governor` without integrating `GovernorTimelockControl`. As a result, successful governance proposals can be executed in the exact same block the voting period concludes, offering depositors zero time window to withdraw assets before critical parameter updates take effect.

Risk

- **Likelihood:** Medium. Affects all executed governance proposals.
- **Impact:** Depositors cannot exit before contentious parameter mutations are enacted.

Proof of Concept Architectural inspection confirms the governor lacks timelock extensions, giving an execution delay of 0 seconds.

Recommended Mitigation Inherit `GovernorTimelockControl` and route governance actions through a `TimelockController` contract with a minimum 2-day execution delay.

5. Low, Informational & Gas Observations

[L-01] Inverted Event Parameters in `VaultGuardians::updateGuardianAndDaoCut`

- **File:** `src/protocol/VaultGuardians.sol:85`
- **Observation:** The event `emit VaultGuardians__FeeUpdated(s_guardianAndDaoCut, newCut)` emits `newCut` in both parameters because `s_guardianAndDaoCut` is updated immediately before the emit statement.
- **Mitigation:** Cache `oldCut = s_guardianAndDaoCut` before updating storage.

[L-02] Misleading Custom Error Selector in `_becomeTokenGuardian`

- **File:** `src/protocol/VaultGuardiansBase.sol:203`
- **Observation:** When an invalid allocation ratio is provided, the contract reverts with `VaultGuardiansBase__CantQuit()`, which is semantically misleading for a deployment failure.
- **Mitigation:** Define and emit `VaultGuardiansBase__InvalidAllocation()`.

[I-01] Unused Import in InvestableUniverseAdapter.sol

- **File:** `src/interfaces/InvestableUniverseAdapter.sol:4`
- **Observation:** Import of `IERC20.sol` is unused in the commented header.
- **Mitigation:** Remove the unused import statement.

[G-01] Redundant Dynamic Storage Write in UniswapAdapter::s_pathArray

- **File:** `src/protocol/investableUniverseAdapters/UniswapAdapter.sol:48`
- **Observation:** Initializing a 2-element storage array causes unnecessary SSTORE operations.
- **Mitigation:** Allocate the path array in memory during swap calls (`address[] memory path = new address[] (2)`).

6. Architectural & Systemic Recommendations

1. **Stateful Invariant Testing in CI/CD:** Maintain and execute the stateful invariant suite (`Handler.t.sol` and `Invariant.t.sol`) in automated pipelines. This immediately surfaces discrepancies such as `INV-03` (`totalAssets() == 0` leading to `maxWithdraw == 0`).
2. **Standard ERC4626 Override Completeness:** Ensure that all entrypoints of the ERC4626 standard (`deposit`, `mint`, `withdraw`, `redeem`) consistently apply protocol access controls, accounting fees, and investment routing.
3. **Decoupled Slippage Protection:** Replace hardcoded zero slippage with dynamic slippage checks or Chainlink oracle price bands for all decentralized exchange integrations.
4. **Governance Timelocks & Grace Periods:** Never allow instantaneous execution of parameter changes in protocols custodying user capital. Implement a minimum 48-hour timelock delay.